

GitHub Actions and CI/CD Pipelines

Automating Build, Test & Deploy Workflows

Chapter Overview

- 01** CI/CD Pipeline Fundamentals
- 02** Pipeline Stages & Benefits
- 03** GitHub Actions Overview
- 04** Workflow Configuration
- 05** Building Your First Pipeline
- 06** Advanced GitHub Actions
- 07** Deploying to the Cloud



CI/CD Pipeline Fundamentals

Section 1



What is CI/CD?

Continuous Integration (CI)

- Developers merge code changes into a shared repository frequently
- Each merge triggers an automated build and test cycle
- Catch integration issues early — before they compound

Continuous Delivery (CD)

- Code is automatically built, tested, and prepared for release
- Deployment to production requires a manual approval step

Continuous Deployment

- Every change that passes all tests is deployed automatically — no human gate



Why CI/CD Matters

Without CI/CD

- Manual builds are slow, error-prone, and inconsistent
- Integration problems discovered late — days or weeks after code was written
- Deployments are risky, stressful events

With CI/CD

- Automated pipelines catch bugs within minutes of a commit
- Deployments become routine — small, frequent, and reversible
- Teams ship faster with higher confidence



The DevOps Connection

DevOps Culture

- Break down silos between Development and Operations teams
- Shared ownership of the entire software lifecycle

CI/CD in DevOps

- CI/CD is the technical backbone of DevOps
- Automates the path from code commit to production deployment
- Enables rapid feedback loops — developers know in minutes if something broke
- Supports infrastructure as code, monitoring, and continuous improvement

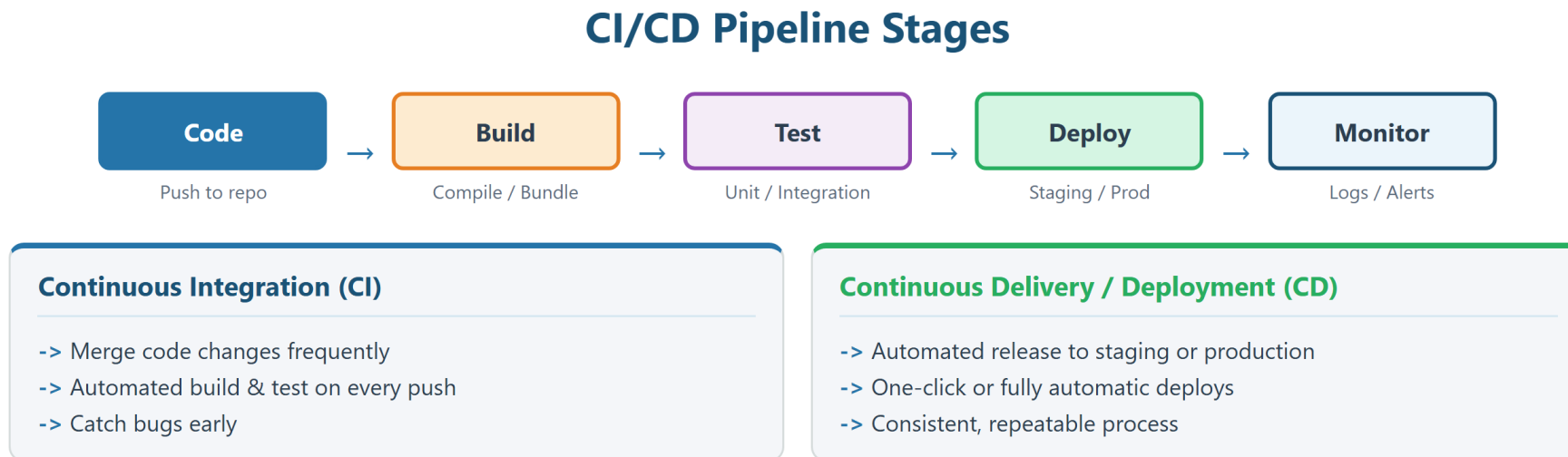


Pipeline Stages & Benefits

Section 2



CI/CD Pipeline Stages



Stage 1 — Source (Code Commit)

The Pipeline Trigger

- A developer pushes code or opens a pull request
- The CI/CD system detects the change and starts the pipeline

What Happens

- Source code is checked out from the repository
- Dependencies are resolved and installed
- The pipeline environment is provisioned (containers, VMs)



Stage 2 — Build

Compiling and Packaging

- Compile source code if applicable (Java, C#, Go)
- Install dependencies from lock files for reproducible builds
- Bundle or package: JS (Webpack, Vite), Python (pip install), Docker image

Why It Matters

- Ensures the code builds cleanly on a clean machine — not just your laptop
- Catches missing dependencies, syntax errors, and configuration issues



Stage 3 — Test

Automated Testing

- Unit tests — verify individual functions and methods
- Integration tests — verify components work together
- End-to-end tests — simulate real user interactions

Quality Gates

- Code coverage thresholds — e.g., require $\geq 80\%$ coverage
- Linting and static analysis — enforce style and detect bugs
- Security scanning — check for known vulnerabilities



Stage 4 — Deploy

Deployment Environments

- Staging — a production-like environment for final validation
- Production — the live environment serving real users

Deployment Strategies

- Rolling update — gradually replace old instances with new ones
- Blue-green — switch traffic from old (blue) to new (green) in one step
- Canary — route a small percentage of traffic to the new version first



Benefits of CI/CD

Benefits of CI/CD



Faster Delivery

Reduced time to market with automated pipelines



Improved Quality

Automated testing catches bugs early



Consistent Releases

Repeatable, reliable deployment process



Developer Productivity

Less manual work, more time for coding



Risk Reduction

Smaller changes = easier rollback & debugging



Visibility

Full audit trail of every build & deploy

CI/CD transforms software delivery from manual, error-prone steps into an automated, reliable pipeline

CI/CD Benefits in Practice

Speed

- Faster feedback — know within minutes if a change breaks something
- Ship features and fixes to users faster

Quality

- Automated tests run on every change — no 'forgot to test' scenarios
- Consistent builds across all environments

Reliability

- Small, incremental deployments are easier to debug and roll back
- Reduced human error in build and deployment processes



GitHub Actions Overview

Section 3



What is GitHub Actions?

Key Features

- CI/CD built directly into GitHub
- Define workflows in YAML files stored in your repository
- Runs on GitHub-hosted or self-hosted runners (Linux, macOS, Windows)
- Triggered by any GitHub event: push, PR, issue, schedule, manual

Why GitHub Actions?

- No separate CI/CD tool to set up — it's already in your repo
- Tight integration with pull requests, issues, and deployments
- Free for public repos; private: 2,000 min (Free), 3,000 (Team), 50,000 (Enterprise)
- Self-hosted runners incur no GitHub usage cost



GitHub Actions — Core Concepts

Workflows

- Automated processes defined in YAML — stored in `.github/workflows/` folder
- A repo can have multiple workflows (CI, deploy, release, etc.)

Events

- Triggers that start a workflow: `push`, `pull_request`, `schedule`, `workflow_dispatch`

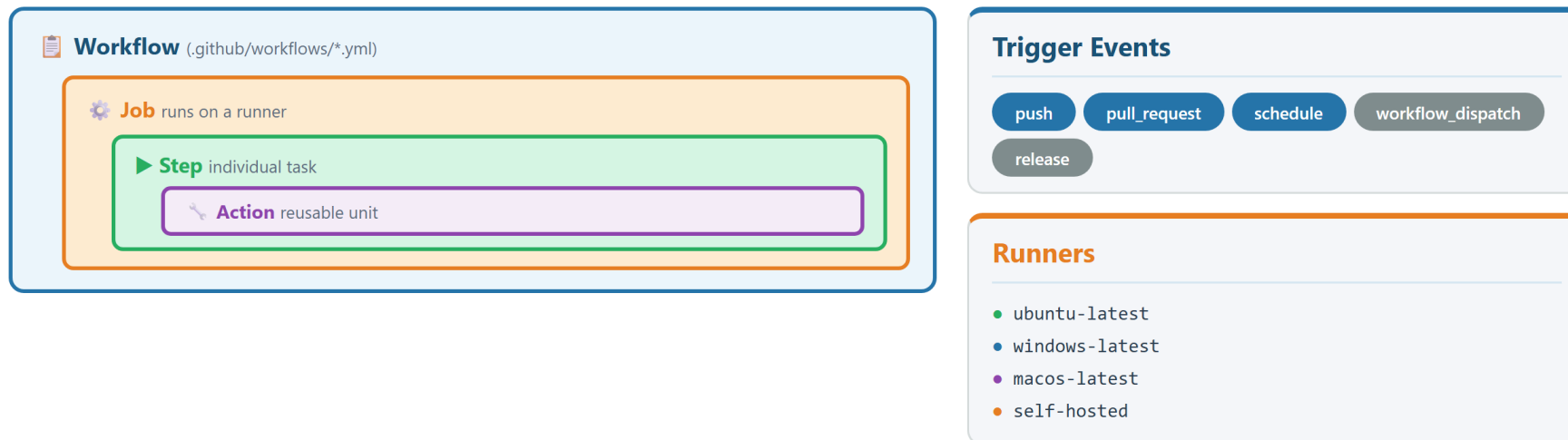
Jobs & Steps

- A workflow contains one or more jobs; each job contains sequential steps
- Jobs run in parallel by default — can define dependencies between them



GitHub Actions Workflow Components

GitHub Actions Components



Workflows automate build, test, and deployment — triggered by repository events

The GitHub Actions Marketplace

Popular Actions

- actions/checkout — check out your repository code
- actions/setup-node — install and configure Node.js
- actions/setup-python — install and configure Python
- actions/cache — cache dependencies for faster builds

Community Actions

- 15,000+ actions available — linting, security scanning, deployment, notifications
- Always pin actions to a specific version: uses: actions/checkout@v5



GitHub Actions vs Other CI/CD Tools

GitHub Actions

- Built into GitHub — no external service needed
- YAML-based, version-controlled alongside your code

Other CI/CD Tools

- Jenkins — open-source, self-hosted, highly customizable but complex
- GitLab CI — similar YAML approach, built into GitLab
- CircleCI, Travis CI — cloud-hosted CI/CD services



Workflow Configuration

Section 4



Anatomy of a Workflow File

Anatomy of a GitHub Actions Workflow

```
name: CI Pipeline          ← workflow name
on:                         ← trigger events
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:                       ← job definitions
  build:
    runs-on: ubuntu-latest  ← runner
    steps:                  ← step list
      - uses: actions/checkout@v5 ← action ref
      - run: npm install
      - run: npm test
```

Keys

YAML property names

Values

Configuration strings

Comments

← Annotations

File Location

.github/workflows/ci.yml

Workflow Triggers (Events)

Push & Pull Request Events

- on: push — run on every push to specified branches
- on: pull_request — run when a PR is opened, updated, or synchronized

Other Triggers

- schedule — cron-based scheduling (e.g., nightly builds)
- workflow_dispatch — manual trigger from the GitHub UI
- release — triggered when a release is published
- repository_dispatch — triggered by external API calls



Basic Workflow — CI Pipeline

```
# .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v6
        with:
          node-version: '20'
      - run: npm ci
      - run: npm test
```



Jobs, Steps & Runners

Jobs

- Each job runs on a fresh virtual machine (runner)
- Jobs run in parallel by default — use 'needs' for sequential execution

Steps

- Sequential tasks within a job — either run: commands or uses: actions
- Steps share the same filesystem and environment within a job

Runners

- GitHub-hosted: ubuntu-latest, windows-latest, macos-latest
- Self-hosted: your own machines for custom environments



Multi-Job Workflow with Dependencies

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - run: npm ci
      - run: npm test

  deploy:
    needs: test          # waits for test to pass
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v5
      - run: npm ci
      - run: npm run build
      - name: Deploy to production
        run: ./deploy.sh
```



Environment Variables & Secrets

Environment Variables

- Set at workflow, job, or step level using env:
- Access with `${{ env.MY_VAR }}` in expressions or `$MY_VAR` in shell

Secrets

- Encrypted values stored in repository or organization settings
- Access with `${{ secrets.MY_SECRET }}` — never printed in logs
- Use for API keys, tokens, passwords, and deployment credentials



Building Your First Pipeline

Section 5



Getting Started — Quick Setup

Step 1: Create the Workflow Directory

- Create `.github/workflows/` in the root of your repository

Step 2: Add a Workflow File

- Create a `.yml` file (e.g., `ci.yml`) inside `.github/workflows/`

Step 3: Push to GitHub

- Commit and push — GitHub automatically discovers and runs the workflow

Step 4: Check the Results

- Go to the Actions tab in your repo to see workflow runs and logs



Your First Workflow — Hello World

```
# .github/workflows/hello.yml
name: Hello World

on:
  push:
    branches: [main]
  workflow_dispatch:      # allows manual trigger

jobs:
  greet:
    runs-on: ubuntu-latest
    steps:
      - name: Say hello
        run: echo 'Hello, GitHub Actions!'

      - name: Show environment
        run: |
          echo "Repository: ${github.repository}"
          echo "Branch: ${github.ref_name}"
```



Testing Workflow — Python Example

```
# .github/workflows/python-ci.yml
name: Python CI

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.11', '3.12']
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-python@v6
        with:
          python-version: ${ matrix.python-version }
      - run: pip install -r requirements.txt
      - run: pytest --cov
```



Debugging & Monitoring Workflows

The Actions Tab

- View all workflow runs — filter by branch, event, or status
- Click into a run to see job logs, timing, and artifacts

Debugging Tips

- Check the logs — expand each step to see detailed output
- Use 'Run workflow' button for manual testing (workflow_dispatch)
- Add debugging steps: run: env to see all environment variables
- Re-run failed jobs from the UI without pushing a new commit



Best Practices for CI/CD Pipelines

Pipeline Design

- Keep pipelines fast — cache dependencies, run tests in parallel
- Fail fast — run quick checks (lint, format) before expensive tests
- Separate CI and CD workflows — keep each small and focused

Security

- Pin third-party actions to a full commit SHA in production
- Use OIDC instead of long-lived credentials for cloud access

Reliability

- Use matrix builds to test across multiple OS/language versions
- Use artifacts to pass data between jobs; add status badges to your README



Advanced GitHub Actions

Section 6



Matrix Builds

What is a Matrix Strategy?

- Run the same job across multiple configurations in parallel
- Combine OS, language version, and other variables into a test grid

Configuration

- `strategy.matrix` — define arrays of values to combine
- Each combination spawns a separate runner in parallel

Control Behavior

- `fail-fast: false` — continue other combinations even if one fails
- `max-parallel: 2` — limit concurrent jobs to control costs



Matrix Build Example

```
jobs:
  test:
    runs-on: ${ matrix.os }
    strategy:
      fail-fast: false
    matrix:
      os: [ubuntu-latest, windows-latest, macos-latest]
      node-version: ['18', '20', '22']
      exclude:
        - os: macos-latest
          node-version: '18'
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v6
        with:
          node-version: ${ matrix.node-version }
      - run: npm ci
      - run: npm test
```

Matrix Builds, Caching & Artifacts

Matrix Builds, Caching & Artifacts

Matrix Strategy

Test across multiple configurations in parallel

OS	Node	Status
ubuntu	18	PASS
ubuntu	20	PASS
windows	18	PASS
windows	20	FAIL

```
strategy:
  matrix:
    os: [ubuntu, windows]
    node: [18, 20]
```

Dependency Caching

Speed up builds by caching downloaded packages

npm ci

→

Cache

→

npm ci

45s (cold)

saved

5s (warm)

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${{ hashFiles('**/package-lock.json') }}
```

Artifacts

Share data between jobs: actions/upload-artifact@v4 & actions/download-artifact@v4



Caching Dependencies

Why Cache?

- npm ci, pip install, and other dependency installs are slow (30-120+ seconds)
- Cache saves downloaded packages between runs — restoring in seconds

How It Works

- actions/cache@v4 — saves a directory keyed by a hash of your lock file
- Cache hit → dependencies restored quickly; install step becomes significantly faster
- Cache miss → install and save for next run

Built-In Caching

- actions/setup-node has a built-in cache option: cache: 'npm'
- actions/setup-python supports cache: 'pip' — same pattern



Caching in Practice

```
# Option 1: Built-in caching with setup actions
- uses: actions/setup-node@v6
  with:
    node-version: '20'
    cache: 'npm'           # caches ~/.npm automatically

# Option 2: Manual cache for custom paths
- uses: actions/cache@v4
  with:
    path: |
      ~/.npm
      node_modules
    key: deps-${{ hashFiles('**/package-lock.json') }}
    restore-keys: deps-

# Cache is scoped to the branch; falls back to the default branch
```



Artifacts — Sharing Data Between Jobs

What Are Artifacts?

- Files produced by one job that other jobs or humans need to access
- Build output, test reports, coverage files, compiled binaries

Upload & Download

- `actions/upload-artifact@v4` — save files from a job
- `actions/download-artifact@v4` — retrieve in a downstream job

Key Details

- Artifacts are retained for 90 days by default (configurable)
- Downloadable from the Actions UI for any workflow run



Artifacts Example — Build and Deploy

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - run: npm ci && npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: dist
          path: ./dist/

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: dist
      - run: ./deploy.sh dist/
```



Expressions & Contexts

Expression Syntax

- `${{ expression }}` — evaluated by GitHub before the step runs
- Use in `if`: conditions, `env`: values, `with`: inputs, and `run`: commands

Common Contexts

- `github.*` — event info: `ref`, `sha`, `actor`, `repository`, `event_name`
- `secrets.*` — encrypted secrets from repo/org settings
- `env.*` — environment variables set at workflow/job/step level
- `matrix.*` — current matrix combination values
- `steps.<id>.outputs.*` — outputs from a previous step



Reusable Workflows & Composite Actions

Reusable Workflows

- Define a workflow that other workflows can call with `workflow_call` trigger
- Pass inputs and secrets — great for org-wide CI/CD standards
- Caller: `uses: org/repo/.github/workflows/ci.yml@main`

Composite Actions

- Bundle multiple steps into a single reusable action
- Defined in an `action.yml` file — `runs: using: 'composite'`
- Publish to the Marketplace or use from a private repo



Reusable Workflow Example

```
# .github/workflows/reusable-ci.yml (the reusable workflow)
on:
  workflow_call:
    inputs:
      node-version:
        type: string
        default: '20'

jobs:
  ci:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v6
        with:
          node-version: ${ inputs.node-version }
      - run: npm ci && npm test

# Caller workflow:  uses: ../github/workflows/reusable-ci.yml
```



Environments & Deployment Protection

GitHub Environments

- Named targets (staging, production) configured in repo Settings → Environments
- Each environment can have its own secrets and variables

Protection Rules

- Required reviewers — one or more people must approve before deploy runs
- Wait timer — enforce a delay (e.g., 15 minutes) before deployment starts
- Branch restrictions — only allow deploys from main or release branches

Usage

- Reference with environment: production in your job definition



Actions Security Best Practices

Actions Security Best Practices

Pin Action Versions

Avoid supply chain attacks by pinning to a commit SHA

```
# Good
uses: actions/checkout@8ade13
# Risky
uses: actions/checkout@main
```

Least-Privilege Permissions

Restrict the GITHUB_TOKEN to only what's needed

```
permissions:
  contents: read
  packages: write
```

Secrets Management

Never hardcode credentials — use encrypted secrets

```
env:
  API_KEY: ${ secrets.API_KEY }
# Masked in logs automatically
```

Environment Protection

Gate production deploys with required reviewers



Audit: Settings → Actions → Audit log

Dependabot: Auto-update action versions



Security Best Practices

Action Supply Chain

- Pin third-party actions to a full commit SHA, not a mutable tag
- Use Dependabot to auto-update action versions with PRs

Least-Privilege Permissions

- Set permissions: at the workflow or job level to restrict GITHUB_TOKEN
- Default to contents: read — only add write scopes when needed

Secret Hygiene

- Rotate secrets regularly; use environment-scoped secrets for production
- Prefer OIDC over long-lived cloud credentials whenever possible



Deploying to the Cloud

Section 7



Cloud Authentication with OIDC

The Problem with Static Credentials

- Long-lived access keys stored as secrets can leak and are hard to rotate
- If a secret is compromised, attackers have persistent cloud access

OIDC — OpenID Connect

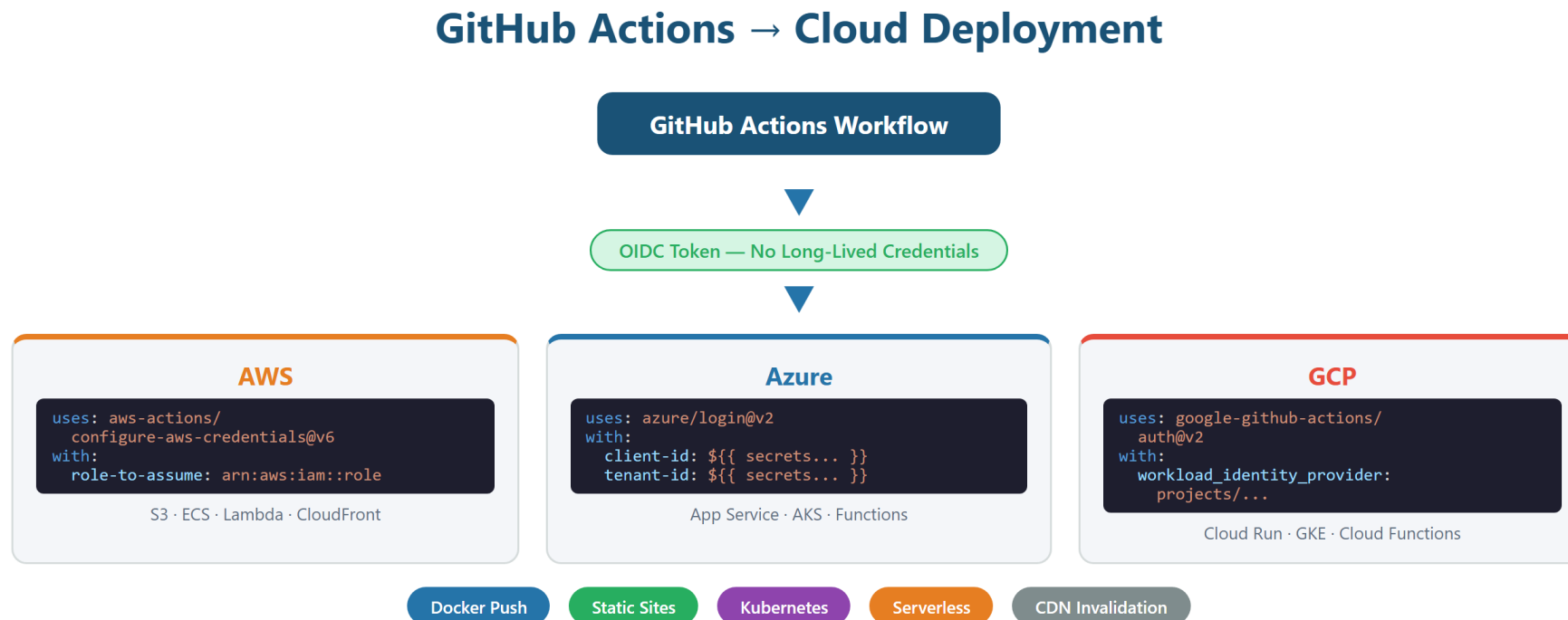
- GitHub Actions requests a short-lived token from your cloud provider
- No secrets to store — the token expires in minutes

How It Works

- Configure a trust relationship between GitHub and your cloud (IAM role / service principal)
- GitHub sends a signed JWT; the cloud verifies it and issues temporary credentials



GitHub Actions → Cloud Deployment



Deploying to AWS

Authentication

- `aws-actions/configure-aws-credentials@v6` — OIDC with an IAM role (recommended)
- Alternative: store `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` as secrets

Common Deployment Targets

- S3 + CloudFront — static site deployment with CDN invalidation
- ECS / Fargate — containerized services with task definition updates
- Lambda — serverless function deployment via ZIP or container image
- Elastic Beanstalk — managed platform for web apps



AWS Deployment — S3 Static Site

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write      # required for OIDC
      contents: read
    steps:
      - uses: actions/checkout@v5
      - uses: aws-actions/configure-aws-credentials@v6
        with:
          role-to-assume: arn:aws:iam::123456789012:role/deploy
          aws-region: us-east-1
      - run: npm ci && npm run build
      - name: Deploy to S3
        run: aws s3 sync dist/ s3://my-bucket --delete
      - name: Invalidate CloudFront
        run: aws cloudfront create-invalidation
              --distribution-id ${ secrets.CF_DIST_ID }
              --paths '/*'
```



Deploying to Azure

Authentication

- azure/login@v2 — OIDC with a service principal and federated credential (verify latest)
- Requires client-id, tenant-id, and subscription-id (not a secret key)

Common Deployment Targets

- App Service — managed web app hosting (azure/webapps-deploy@v3)
- Azure Kubernetes Service (AKS) — container orchestration
- Azure Functions — serverless compute
- Azure Static Web Apps — built-in GitHub Actions integration



Azure Deployment — App Service

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    permissions:
      id-token: write
      contents: read
    steps:
      - uses: actions/checkout@v5
      - uses: azure/login@v2
        with:
          client-id: ${ secrets.AZURE_CLIENT_ID }
          tenant-id: ${ secrets.AZURE_TENANT_ID }
          subscription-id: ${ secrets.AZURE_SUB_ID }
      - run: npm ci && npm run build
      - uses: azure/webapps-deploy@v3
        with:
          app-name: my-web-app
          package: ./dist
```



Deploying to GCP

Authentication

- google-github-actions/auth@v2 — OIDC with Workload Identity Federation
- No service account key files needed — uses short-lived tokens

Common Deployment Targets

- Cloud Run — fully managed container platform (google-github-actions/deploy-cloudrun@v2)
- Google Kubernetes Engine (GKE) — managed Kubernetes clusters
- Cloud Functions — event-driven serverless functions
- Firebase Hosting — fast CDN for static and dynamic web apps



Container Registries & Docker

Build and Push Docker Images

- `docker/build-push-action@v6` — build multi-platform images in CI
- `docker/login-action@v3` — authenticate to any container registry

Registry Options

- GitHub Container Registry (`ghcr.io`) — free for public packages, built into GitHub
- Amazon ECR — private registry integrated with ECS/EKS
- Azure Container Registry — private registry for AKS and App Service
- Google Artifact Registry — private registry for Cloud Run and GKE



Docker Build & Push to GHCR

```
jobs:
  docker:
    runs-on: ubuntu-latest
    permissions:
      packages: write
      contents: read
    steps:
      - uses: actions/checkout@v5
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${GITHUB_ACTOR}
          password: ${GITHUB_TOKEN}
      - uses: docker/build-push-action@v6
        with:
          push: true
          tags: ghcr.io/${GITHUB_REPOSITORY}:latest
```



Cloud Deployment Best Practices

Authentication

- Always prefer OIDC over static credentials — no secrets to rotate or leak
- Scope IAM roles to the minimum permissions needed for the deployment

Deployment Strategy

- Use environments (staging → production) with protection rules and approvals
- Tag Docker images with the commit SHA, not just 'latest'

Reliability

- Add health checks after deployment to verify the new version is running
- Automate rollback — if health checks fail, redeploy the previous version



Modern CI/CD

Security & Supply Chain

- OIDC-based authentication is the standard — static credentials are legacy
- Shift-left security: SAST, secrets scanning, and dependency audits run in CI
- SBOM generation and artifact signing provide provenance and tamper detection
- Artifact provenance (SLSA) and signed builds becoming standard in enterprise pipelines

Developer Experience

- Ephemeral environments spin up per PR — review live previews before merging
- AI-assisted pipeline optimization suggests caching, parallelization, and fixes
- Reusable workflows and composite actions are the norm — DRY across repos



Key Takeaways

Key Takeaways

1. CI/CD automates the build, test, and deploy cycle — catching bugs early and shipping faster
2. GitHub Actions workflows are YAML files in `.github/workflows/` triggered by events like `push` and `pull_request`
3. Matrix builds test across OS/version combinations; caching and artifacts speed up pipelines
4. Reusable workflows and composite actions keep CI/CD DRY across repositories
5. Use OIDC for secure, credential-free authentication to AWS, Azure, and GCP
6. Environments with protection rules gate production deployments behind manual approvals

